



Web Development Foundations

Week 4 - JavaScript: "This is Programming"
Day 2: "JS Logic Iteration"

1



Reconnect

Monday:

- ▶ variables stored values
- ▶ console output made values visible
- ▶ 'if / else' chose a path

Today:

- ▶ the same decision becomes reusable
- ▶ functions help organize logic

2



From Monday to Wednesday

Your decision logic becomes a reusable function.

1 Monday: Variables and if/else

```
let score = 85;
let onTime = true;
if (score >= 80 && onTime) {
  console.log("on track");
} else {
  console.log("needs review");
}
```

 Decision logic works. But it's written inline.

2 Tuesday: Named function

```
function checkStatus(score, onTime) {
  if (score >= 80 && onTime) {
    console.log("on track");
  } else {
    console.log("needs review");
  }
}
```

 The same logic is grouped and named.

3 Wednesday: Function call and console result

```
let score = 85;
let onTime = true;
checkStatus(score, onTime);
// Console output: on track
```

 Call the function. Read the result.

 **Same decision. Clearer organization.**

Reconnect

3



The Working Problem: Logic Gets Repeated Or Messy

Working code can still be hard to follow.

Common signs:

- ▶ names are vague
- ▶ output is unclear
- ▶ the same decision appears more than once
- ▶ changing one value is confusing
- ▶ the result is hard to explain

4



Working JavaScript Becomes Easier to Follow

Same result. Better structure.

working but hard to follow

```
let s = 85;
let t = true;
if (s >= 80 && t) {
  console.log("on track");
} else {
  console.log("needs review");
}

let s2 = 87;
let t2 = false;
if (s2 >= 80 && t2) {
  console.log("on track");
} else {
  console.log("needs review");
}
```

 Same logic repeated.

clearer function

```
function checkStatus(score, onTime) {
  if (score >= 80 && onTime) {
    console.log("on track");
  } else {
    console.log("needs review");
  }
}

let s = 85;
let t = true;
checkStatus(s, t);

let s2 = 87;
let t2 = false;
checkStatus(s2, t2);
```

 Named once. Used many times.

 **Working code can still be improved.**
Small changes in structure make code easier to read and reuse.

Messy Logic -> Clearer Logic

5



What Counts as Success Today

- a function has one clear job
- inputs are named clearly
- the function is called
- 'return' sends a result back
- console output is explainable

6

Functions Package A Small Job

A function packages steps under a name.

Use a function when:

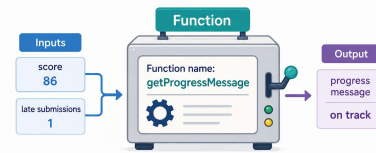
- ▶ a task has a clear purpose
- ▶ the logic may be reused
- ▶ you want code to read like an idea

Good function names explain the job.

7

A Function Is a Named Package for a Small Job

It takes inputs, does its job, and gives back a result.



Good function names explain the job.
A clear name makes your code easier to read and reuse.

Function Packages A Small Job

8

What We Will Use Today

- ▶ function
- ▶ parameter
- ▶ argument
- ▶ 'return'
- ▶ function call
- ▶ meaningful names
- ▶ console output
- ▶ predict, run, observe

9

Today's JavaScript Toolbox

These tools help us build and organize logic with functions.



Simple tools. Strong results.
Use these tools with purpose, and your code will be clearer, easier to follow, and easier to reuse.

Practice today. Apply everywhere.

Function Toolbox

10

Define Versus Call

Define:

"Here are the steps."

Call:

"Run those steps now."

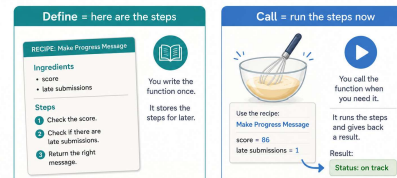
Example:

```
1 function getProgressMessage(scoreVar, lateSubmissionsVar) {
2   // code goes here
3 }
4
5 const message = getProgressMessage(score, lateSubmissions);
6
```

11

Define vs. Call for Functions

Two different actions. One goal.



A function does useful work when it is called.
Define once. Call as many times as you need.

Define vs. Call

12



Return Sends A Value Back

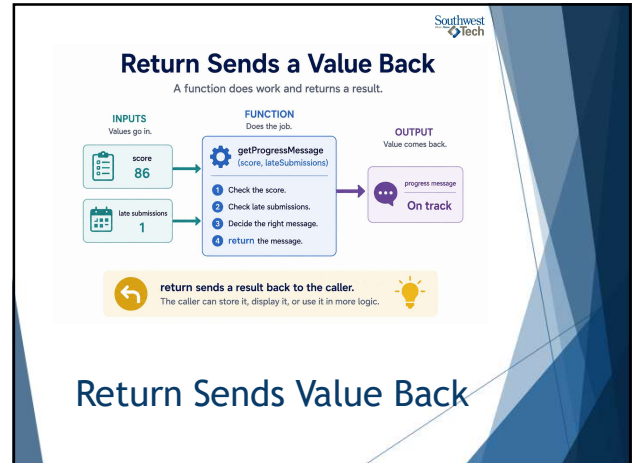
`'return'` sends a result back to the caller.

The function can decide:

- ▶ "On track"
- ▶ "Needs attention"
- ▶ "Schedule support"

Then another line can store or print that result.

13



Return Sends a Value Back

A function does work and returns a result.

INPUTS
Values go in.

FUNCTION
Does the job.

OUTPUT
Value comes back.

score: 86
late submissions: 1

`getProgressMessage(score, lateSubmissions)`

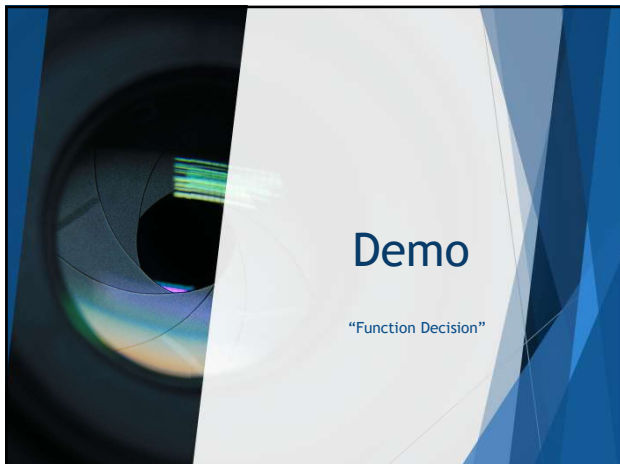
- 1 Check the score.
- 2 Check late submissions.
- 3 Decide the right message.
- 4 return the message.

progress message: On track

return sends a result back to the caller.
The caller can store it, display it, or use it in more logic.

Return Sends Value Back

14



Demo

"Function Decision"

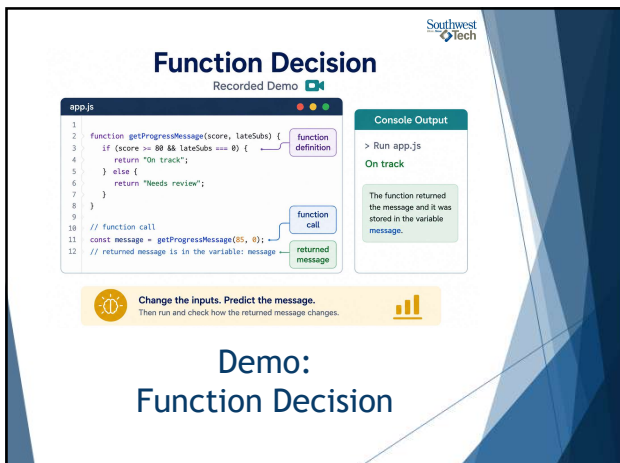
15



What to Watch For

- ▶ function definition
- ▶ parameters as inputs
- ▶ `'if'` decisions inside the function
- ▶ `'return'` message
- ▶ function call
- ▶ console output after the call

16



Function Decision

Recorded Demo

```

1 function getProgressMessage(score, lateSubs) {
2   if (score >= 80 && lateSubs === 0) {
3     return "On track";
4   } else {
5     return "Needs review";
6   }
7 }
8
9 // function call
10 const message = getProgressMessage(85, 0);
11 // returned message is in the variable: message
12 
```

Console Output

```

> Run app.js
On track

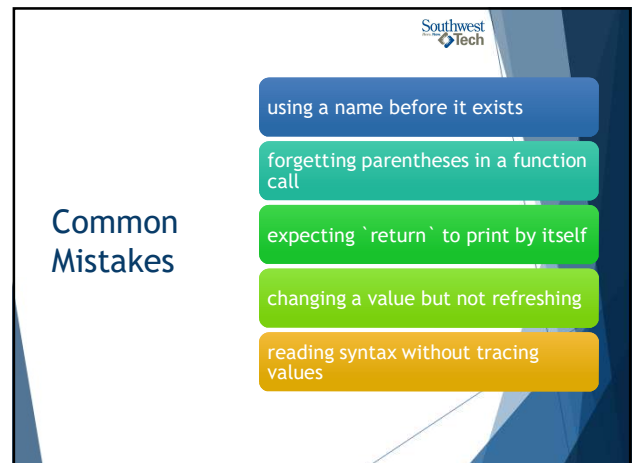
```

The function returned the message and it was stored in the variable message.

Change the inputs. Predict the message.
Then run and check how the returned message changes.

Demo: Function Decision

17



Common Mistakes

- using a name before it exists
- forgetting parentheses in a function call
- expecting `'return'` to print by itself
- changing a value but not refreshing
- reading syntax without tracing values

18



Common Beginner JavaScript Logic Mistakes

A friendly checklist to keep your code on track.

- 1 name used before it exists**
Make sure the variable or function is declared before you use it.
- 2 missing function parentheses**
Remember to add () when calling a function.
- 3 return does not print by itself**
return sends a value back, but it does not display it.
- 4 value changed but page not refreshed**
Browser pages don't update automatically. Refresh or return to see changes.
- 5 syntax read without tracing values**
Reading code is not enough. Trace the values step by step.

Slow down and trace the values.
Understanding comes from watching the values as they move.

Common Logic Mistakes

19



Lab Refinement

Improve the program.

Your goal:

- ▶ use meaningful names
- ▶ add or improve one function
- ▶ improve or expand one condition
- ▶ keep console output understandable
- ▶ make the logic easier to explain

20



Evidence & Reflection

Final Assignment 4 evidence:

- ▶ '.js' file only
- ▶ code runs without errors
- ▶ output appears in the console
- ▶ variables, condition, and function are present

Reflection:

- ▶ Explains what felt different from HTML/CSS

21



How To Read Next Week's Material



Required:

- read for the big idea of the DOM as a bridge
- notice how JavaScript can find one page element
- notice basic click-event thinking

Skim:

- do not memorize every DOM method or event type
- look for examples that feel close to buttons and page text

Reference:

- jQuery is recognition and helper-library awareness, not the main model

Before next time:

- ▶ Bring one question about how code connects to visible page behavior.

22



JavaScript Logic Preparing to Connect to the DOM Next Week

We build the logic first. Then we connect it to the page.

JavaScript Logic
program runs

Your JavaScript makes decisions, calculates results, and produces messages.

- variables
- if/else decisions
- functions
- return values

HTML Page
page structure

Your HTML creates the structure and places where information will appear.

- headings
- paragraphs
- lists
- sections

DOM - next week

The page is ready. The program is ready.
Next, they meet.

JavaScript To DOM Bridge

23



Closing Success Check

This week:

- ▶ JavaScript became a programming language
- ▶ values changed program behavior
- ▶ conditions chose paths
- ▶ functions organized logic

Next:

- ▶ JavaScript connects to HTML through the DOM.

24



25